*TP n° 1*

# Virtualisation avec KVM/QEMU et libvirt

VM • Clones • Images d'OS • Templates

## Fiche pédagogique

**Titre : Créer, dupliquer et industrialiser des machines virtuelles avec KVM/QEMU et libvirt**

**Durée conseillée : 6 à 7 heures, réparties en deux séances**, hors téléchargement des ISO. Travail individuel, avec validation pratique à chaque étape.

**Environnements :** Ubuntu 24.04 LTS ou Rocky Linux 9, sur architecture x86_64. Choisir un seul parcours pour l'ensemble du TP.

**Situation professionnelle :** vous devez installer un serveur Linux, reproduire son environnement pour un test, conserver une image réutilisable de son système et préparer une référence permettant de déployer plusieurs serveurs aux identités distinctes.

> Le TP est réussi lorsque l'étudiant sait créer une VM, réaliser un clone complet, capturer et réimporter une image de système installé, préparer un template généralisé et déployer deux VM distinctes à partir de ce template. Il doit également justifier le choix entre clone, template et image dans une situation donnée.

## 1. Compétences et preuves attendues

| À la fin du TP, l'étudiant doit savoir… | Preuve observable |
|---|---|
| Vérifier la disponibilité de KVM et préparer libvirt | Diagnostic matériel et connexion à qemu:///system |
| Créer une VM à partir d'une ISO | VM installée, démarrée et accessible |
| Administrer le cycle de vie d'une VM | Démarrage, arrêt propre et inspection de sa configuration |
| Créer un clone complet | Nouvelle définition libvirt et disque indépendant |
| Identifier les identités dupliquées par clonage | Comparaison MAC, UUID libvirt, hostname, machine-id et clés SSH |
| Capturer une image d'un système installé | Image QCOW2 contrôlée, documentée et réimportée avec succès |

| À la fin du TP, l'étudiant doit savoir… | Preuve observable |
|---|---|
| Généraliser une référence système | Nettoyage justifié et référence conservée hors exécution |
| Déployer un template | Deux instances ayant des identités système distinctes |
| Choisir la bonne méthode | Réponses argumentées à des cas professionnels |

# 2. Comprendre avant de manipuler

## 2.1 VM, image, clone et template : quatre notions liées

**Une VM** est un ordinateur virtuel défini par une configuration matérielle : CPU, RAM, interfaces réseau, firmware et disques. Dans libvirt, cette définition peut être exportée au format XML.

**Une image disque** est un fichier représentant le contenu d'un disque virtuel. Elle peut être vide, contenir uniquement des données ou contenir un système d'exploitation installé. RAW et QCOW2 sont des formats d'images disque.

**Un clone** est une nouvelle VM obtenue par duplication d'une VM existante. Un clone complet possède ses propres disques. L'outil de clonage peut renouveler les identifiants matériels virtuels sans renouveler toutes les identités internes du système invité.

**Un template**, ou modèle de référence, est une base préparée pour des déploiements répétables : OS, logiciels communs, configuration commune, nettoyage des éléments propres à l'instance et mécanisme de personnalisation au premier démarrage.

> Ces notions ne s'excluent pas : un template peut être stocké sous forme d'image QCOW2, et une VM peut être créée par clonage de cette référence. « Image » décrit un support ; « template » décrit un rôle et un niveau de préparation ; « clone » décrit une opération et son résultat.

## 2.2 Ne pas confondre les différentes images

| Objet | Contenu | Usage |
|---|---|---|
| ISO d'installation | Installateur et fichiers d'installation | Installer un système sur un disque virtuel |
| Image disque vide | Capacité de stockage sans OS installé | Ajouter un disque ou préparer une installation |
| Image d'un système installé | Partitions, OS, logiciels et éventuellement données | Capturer un état, transférer ou redéployer un système compatible |

| Objet | Contenu | Usage |
|---|---|---|
| Image cloud officielle | OS préinstallé, généralement préparé pour cloud-init | Déployer sans installation interactive, après configuration initiale |
| Image de référence généralisée | OS commun préparé pour plusieurs futures instances | Servir de template |

**Créer un fichier QCOW2 vide ne signifie pas créer une image d'OS utilisable.** Ce TP impose donc de capturer un système installé, puis de vérifier qu'une VM peut réellement démarrer depuis cette image.

### 2.3 Quand utiliser chaque approche ?

| Besoin | Choix généralement adapté | Précaution |
|---|---|---|
| Reproduire rapidement un environnement pour tester une modification | Clone | Isoler les données sensibles et vérifier les identités |
| Déployer régulièrement des serveurs standardisés | Template versionné et personnalisé | Généraliser puis valider chaque version |
| Conserver ou transférer le contenu d'un disque système | Image disque | Conserver les paramètres nécessaires à la restauration |
| Installer un OS vierge manuellement | ISO d'installation | L'ISO n'est pas le disque système de la VM finale |
| Revenir rapidement avant une manipulation locale | Snapshot, si adapté | Un snapshot n'est pas une sauvegarde indépendante |
| Sauvegarder un service de production | Procédure de sauvegarde vérifiée | Une image seule ne garantit ni cohérence applicative ni restauration complète |

**KVM** fournit l'accélération de virtualisation dans le noyau Linux. **QEMU** fournit le modèle de machine et les périphériques virtuels ; il peut utiliser KVM pour accélérer l'exécution. **libvirt** gère les VM, réseaux et stockages ; `virsh`, `virt-install` et `virt-manager` utilisent cet environnement de gestion.

## 3. Prérequis, conventions et sécurité

- PC x86_64 avec Intel VT-x ou AMD-V activé dans l'UEFI/BIOS.
- 16 Go de RAM recommandés ; avec 8 Go, exécuter les VM successivement.
- Environ 60 Go d'espace libre, à adapter au nombre de copies et à leur occupation réelle.

- Ubuntu 24.04 LTS ou Rocky Linux 9 installé sur l'hôte.
- Si l'hôte est virtualisé : activer la virtualisation imbriquée dans l'hyperviseur parent.
- Connexion Internet pour les paquets et les ISO.
- Interface graphique recommandée pour suivre l'installation avec `virt-manager`.
- Le parcours utilise le réseau NAT `default` et la connexion `qemu:///system`.
- Les exemples supposent un disque système unique, non chiffré, sans TPM ni Secure Boot imposé. Un système UEFI exige de conserver aussi les paramètres de firmware et, selon le cas, la NVRAM.

**Toutes les commandes sont exécutées sur l'hôte, sauf les sections explicitement marquées « DANS LA VM ».** Les variables définies sur l'hôte ne sont pas automatiquement présentes dans la VM.

Règles impératives :

- Ne jamais copier, convertir, généraliser ou modifier hors ligne un disque utilisé par une VM en cours d'exécution.
- Attendre l'arrêt effectif : `virsh shutdown` ne bloque pas jusqu'à la fin de l'arrêt.
- Ne jamais généraliser l'unique copie d'un disque important.
- Ne jamais attacher le même disque inscriptible à deux VM simultanément.
- Ne pas réutiliser un chemin de sortie existant : choisir un nouveau nom plutôt que l'écraser.
- Ne pas inclure de secrets, clés privées personnelles, jetons ou données de production dans une référence.
- Ne pas confondre clé SSH **hôte**, qui identifie un serveur, et clé SSH **utilisateur**, qui authentifie une personne.
- Ne pas désactiver SELinux, AppArmor ou le pare-feu pour contourner un problème de configuration.

# Partie A — Préparer l'environnement · 40 min

## Exercice 1 — Vérifier la virtualisation

**SUR L'HÔTE :**

```
lscpu
grep -Eoc '(vmx|svm)' /proc/cpuinfo
lsmod | grep kvm
test -e /dev/kvm && ls -l /dev/kvm
df -h /var/lib/libvirt
```

Si le répertoire libvirt n'existe pas encore, vérifier l'espace sur `/var`.

**À relever :**

- Architecture et modèle du processeur.
- Signification de `vmx` et `svm`.
- Présence de `/dev/kvm` et des modules `kvm_intel` ou `kvm_amd`.
- RAM et espace disque disponibles.

Un compteur non nul indique que les extensions sont exposées à l'OS ; ce n'est pas, à lui seul, une validation complète du fonctionnement de KVM. Un compteur nul nécessite de vérifier le firmware ou la virtualisation imbriquée.

## Exercice 2 — Installer et contrôler les outils

**Hôte Ubuntu 24.04 :**

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients \
  virtinst virt-manager virt-viewer qemu-utils libguestfs-tools genisoimage
sudo systemctl enable --now libvirtd
sudo usermod -aG libvirt,kvm "$USER"
```

**Hôte Rocky Linux 9 :**

```
sudo dnf install -y qemu-kvm libvirt virt-install virt-manager \
  virt-viewer qemu-img libguestfs-tools genisoimage
sudo systemctl enable --now libvirtd
sudo usermod -aG libvirt "$USER"
```

Déconnecter puis reconnecter la session pour actualiser les groupes. Selon les paquets et la configuration, libvirt peut utiliser des démons modulaires activés par sockets. Si `libvirtd` n'est pas disponible, contrôler notamment `virtqemud.socket` et faire valider la configuration par l'enseignant.

```
sudo virsh -c qemu:///system list --all
command -v virt-install
command -v virt-clone
command -v qemu-img
command -v virt-sysprep
command -v genisoimage
```

Sur Ubuntu, `virt-clone` est fourni par `virtinst` ; sur Rocky, par `virt-install`. Si un paquet ou une commande manque, rechercher son fournisseur et les dépôts nécessaires ; ne pas poursuivre avec un outil absent.

**Validation A :** libvirt répond et tous les outils nécessaires sont disponibles.

### Préparer les variables de travail

Dans un terminal **Bash sur l'hôte**, choisir un seul bloc et le conserver pour la suite.

**Parcours Ubuntu :**

```
export PREFIX=ubuntu
export OSVAR=ubuntu24.04
export ISO=/var/lib/libvirt/boot/ubuntu.iso
```

**Parcours Rocky :**

```
export PREFIX=rocky9
export OSVAR=rocky9
export ISO=/var/lib/libvirt/boot/rocky9.iso
```

**Puis, dans les deux parcours :**

```
export VM="${PREFIX}-vm01"
export CLONE="${PREFIX}-vm02"
export RESTORE="${PREFIX}-restore01"
export IMGDIR=/var/lib/libvirt/images
export DISK="$IMGDIR/$VM.qcow2"
export CAPTURE="$IMGDIR/${PREFIX}-os-capture-v1.qcow2"
export TEMPLATE="$IMGDIR/${PREFIX}-template-v1.qcow2"

mkdir -p "$HOME/tp1-kvm/preuves"
sudo mkdir -p "$IMGDIR" /var/lib/libvirt/boot

printf 'VM=%s\nCLONE=%s\nDISK=%s\n' "$VM" "$CLONE" "$DISK"
```

En cas de nouveau terminal, redéfinir ces variables. Ne pas exécuter les commandes avec des variables vides.

## Partie B — Créer et administrer une VM · 75 min

### Exercice 3 — Préparer l'ISO et le réseau

Télécharger une ISO officielle Ubuntu 24.04 Server ou Rocky Linux 9 Minimal et vérifier sa somme de contrôle selon la procédure publiée par la distribution.

```
# Adapter le chemin réel du téléchargement.
sudo cp /chemin/vers/image.iso "$ISO"

sudo virsh -c qemu:///system net-list --all
```

Si `default` existe mais est inactif :

```
sudo virsh -c qemu:///system net-start default
```

S'il existe, activer son démarrage automatique :

```
sudo virsh -c qemu:///system net-autostart default
```

Si `default` n'existe pas, demander à l'enseignant de fournir ou définir le réseau NAT avant de poursuivre. Vérifier que l'ISO est lisible par les processus de virtualisation.

**Question :** pourquoi une VM derrière le NAT peut-elle généralement accéder à Internet sans être directement accessible depuis tous les postes du réseau physique ?

## Exercice 4 — Installer la VM

Configuration cible :

- 2 vCPU ;
- 2 048 Mio de RAM ;
- disque QCOW2 de 15 Gio ;
- disque et réseau VirtIO ;
- réseau NAT `default`.

Vérifier la variante prise en charge :

```
virt-install --osinfo list | grep -Ei 'ubuntu|rocky'
```

Si la variante attendue est absente, mettre à jour les données OS ou choisir une variante compatible validée par l'enseignant.

```
sudo virt-install \
  --connect qemu:///system \
  --name "$VM" \
  --memory 2048 \
  --vcpus 2 \
  --disk path="$DISK",size=15,format=qcow2,bus=virtio \
  --cdrom "$ISO" \
  --network network=default,model=virtio \
  --channel unix,target_type=virtio,name=org.qemu.guest_agent.0 \
  --os-variant "$OSVAR" \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Ouvrir `virt-manager`, sélectionner la connexion système puis la console de la VM. L'affichage VNC reste limité à l'hôte ; ne pas l'exposer sans protection sur le réseau.

Pendant l'installation :

- attribuer le nom d'hôte `srv-source` ;
- créer un compte pédagogique local sans donnée personnelle ;
- utiliser DHCP, sans adresse statique ni liaison à une MAC particulière ;
- installer le serveur SSH ;
- choisir un partitionnement simple sans chiffrement pour les outils hors ligne du TP ;
- après installation, éjecter l'ISO si nécessaire et démarrer sur le disque.

## Exercice 5 — Administrer et personnaliser la source

**SUR L'HÔTE :**

```
sudo virsh -c qemu:///system list --all
sudo virsh -c qemu:///system dominfo "$VM"
sudo virsh -c qemu:///system domblklist "$VM"
sudo virsh -c qemu:///system domiflist "$VM"
sudo virsh -c qemu:///system dumpxml "$VM" \
  > "$HOME/tp1-kvm/preuves/source.xml"
```

**DANS LA VM — Ubuntu :**

```
sudo apt update
sudo apt install -y cloud-init qemu-guest-agent openssh-server
sudo systemctl enable --now ssh
sudo systemctl start qemu-guest-agent
```

**DANS LA VM — Rocky :**

```
sudo dnf install -y cloud-init qemu-guest-agent openssh-server
sudo systemctl enable --now sshd
sudo systemctl start qemu-guest-agent
```

Le canal VirtIO de l'agent invité a été ajouté lors de la création de la VM. Selon la distribution, le service de l'agent est activé par le périphérique et ne nécessite pas forcément `systemctl enable`. Contrôler son état plutôt que considérer un message « static » comme un échec.

**DANS LA VM — commandes communes :**

```
sudo mkdir -p /opt/tp1
echo "Version applicative TP1 : v1" | sudo tee /opt/tp1/version.txt
hostnamectl
cat /etc/machine-id
ip -br addr
cat /opt/tp1/version.txt
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
systemctl status qemu-guest-agent --no-pager
```

Si la clé Ed25519 n'existe pas, relever l'empreinte d'une autre clé hôte présente et utiliser le même type de clé pour toutes les comparaisons.

**SUR L'HÔTE :**

```
sudo virsh -c qemu:///system domifaddr "$VM" --source agent
sudo virsh -c qemu:///system net-dhcp-leases default
```

Une sortie vide de `domifaddr` ne prouve pas l'absence de réseau : utiliser la console, vérifier l'agent puis les baux DHCP.

Tester un arrêt et un redémarrage :

```
sudo virsh -c qemu:///system shutdown "$VM"
sudo virsh -c qemu:///system domstate "$VM"
# Attendre l'arrêt effectif avant la commande suivante.
sudo virsh -c qemu:///system start "$VM"
```

**Questions :**

1. Quelle différence entre `shutdown` et `destroy` ?
2. Pourquoi `destroy` ne signifie-t-il pas « supprimer le disque » ?
3. Que supprime `undefine` et que conserve-t-il normalement ?
4. À quoi servent les pilotes VirtIO ?
5. Pourquoi le XML seul ne suffit-il pas à reconstituer la VM ?

**Validation B :** VM opérationnelle, fichier témoin présent, réseau fonctionnel et configuration enregistrée.

# Partie C — Cloner et observer les identités · 50 min

## Exercice 6 — Créer un clone complet

**SUR L'HÔTE :**

```
sudo virsh -c qemu:///system shutdown "$VM"
sudo virsh -c qemu:///system domstate "$VM"
```

Attendre l'état arrêté, souvent affiché `shut off` selon la langue. Ne pas poursuivre si la VM tourne encore.

```
sudo virt-clone \
  --connect qemu:///system \
  --original "$VM" \
  --name "$CLONE" \
  --file "$IMGDIR/$CLONE.qcow2"

sudo virsh -c qemu:///system domuuid "$VM"
sudo virsh -c qemu:///system domuuid "$CLONE"
sudo virsh -c qemu:///system domiflist "$VM"
sudo virsh -c qemu:///system domiflist "$CLONE"
sudo virsh -c qemu:///system domblklist "$CLONE"
```

Vérifier, VM arrêtées :

```
sudo qemu-img info "$IMGDIR/$CLONE.qcow2"
```

Le clone complet ne doit pas dépendre du disque de la source par un `backing file`.

Démarrer les VM simultanément si les ressources le permettent, sinon successivement :

```
sudo virsh -c qemu:///system start "$VM"
sudo virsh -c qemu:///system start "$CLONE"
```

## Observer avant de corriger

**DANS CHAQUE VM :**

```
hostnamectl --static
cat /etc/machine-id
ip -br addr
cat /opt/tp1/version.txt
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Compléter avec les valeurs réellement observées :

| Élément | Source | Clone | Identique ou différent ? | Interprétation |
|---|---|---|---|---|
| Nom de la VM dans libvirt | À relever | À relever | À établir | Gestion de la VM |
| UUID libvirt | À relever | À relever | À établir | Identité de la définition |
| Adresse MAC | À relever | À relever | À établir | Identité réseau virtuelle |

| Élément | Source | Clone | Identique ou différent ? | Interprétation |
|---|---|---|---|---|
| Adresse IP et moment du relevé | À relever | À relever | À établir | Dépend du réseau et du DHCP |
| Nom d'hôte dans Linux | À relever | À relever | À établir | Identité interne de l'OS |
| machine-id | À relever | À relever | À établir | Identité du système invité |
| Empreinte de clé SSH hôte | À relever | À relever | À établir | Identité cryptographique du serveur |
| Fichier `/opt/tp1/version.txt` | À relever | À relever | À établir | Contenu copié |

**Ne pas supposer que deux MAC différentes suffisent à garantir deux IP différentes.** Les identifiants DHCP et la configuration copiée peuvent influencer les baux. Deux VM testées successivement peuvent aussi recevoir la même IP.

### Prouver l'indépendance des disques

Dans le clone seulement :

```
echo "Modification propre au clone" | sudo tee /opt/tp1/clone.txt
```

Dans la source :

```
test ! -e /opt/tp1/clone.txt && echo "Disques indépendants : contrôle réussi"
```

**À expliquer :**

- Pourquoi un clone peut-il avoir une nouvelle MAC mais conserver le même `machine-id` ?
- Quels risques présente la duplication de clés SSH hôte ?
- Pourquoi faut-il isoler un clone de production avant de le démarrer ?
- Pourquoi un clone complet n'est-il pas automatiquement un template ?

**Validation C :** clone démarrable, disque indépendant et diagnostic des identités dupliquées. Arrêter le clone avant de poursuivre.

## Partie D — Créer et réutiliser une image d'OS · 55 min

### Exercice 7 — Capturer le système installé

Cette étape ne consiste pas à créer un disque vide : elle conserve le système installé et le fichier témoin.

**SUR L'HÔTE :**

```
sudo virsh -c qemu:///system shutdown "$VM"
sudo virsh -c qemu:///system domstate "$VM"
```

Après confirmation de l'arrêt, et uniquement si le chemin de sortie n'existe pas :

```
sudo qemu-img convert -p -f qcow2 -O qcow2 "$DISK" "$CAPTURE"
sudo qemu-img info "$CAPTURE"
sudo qemu-img check -f qcow2 "$CAPTURE"

sudo sha256sum "$CAPTURE" \
  | tee "$HOME/tp1-kvm/preuves/capture.sha256"

sudo virsh -c qemu:///system dumpxml "$VM" \
  > "$HOME/tp1-kvm/preuves/capture-config.xml"
```

Créer une fiche d'image mentionnant :

- distribution, version et architecture ;
- date et identifiant de version ;
- format et capacité virtuelle ;
- logiciels installés et rôle prévu ;
- type de firmware, contrôleurs de disque et configuration réseau ;
- somme de contrôle ;
- statut : **capture non généralisée** ;
- présence éventuelle de comptes et de données ;
- procédure de réimportation.

**Important :** le contrôle structurel et la somme de contrôle ne prouvent pas que le système peut démarrer ni que les applications sont cohérentes. Le test de réimportation est obligatoire. Pour un service réel, une sauvegarde cohérente et testée reste nécessaire.

## Exercice 8 — Réimporter et tester l'image

Ne pas démarrer directement l'image d'archive : créer un disque de travail indépendant.

```
sudo qemu-img convert -p -f qcow2 -O qcow2 \
  "$CAPTURE" "$IMGDIR/$RESTORE.qcow2"

sudo virt-install \
  --connect qemu:///system \
  --name "$RESTORE" \
  --memory 2048 \
  --vcpus 2 \
  --import \
  --disk path="$IMGDIR/$RESTORE.qcow2",format=qcow2,bus=virtio \
  --network network=default,model=virtio \
  --channel unix,target_type=virtio,name=org.qemu.guest_agent.0 \
  --os-variant "$OSVAR" \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Conserver le même mode de firmware que la source. Si celle-ci utilise UEFI, adapter la définition au lieu de supposer que le disque suffit.

**DANS LA VM RÉIMPORTÉE :**

```
cat /etc/os-release
cat /opt/tp1/version.txt
hostnamectl --static
cat /etc/machine-id
```

**Résultat attendu :** le système démarre sans installation interactive et retrouve le fichier témoin. Les identités internes peuvent être celles de la source, car l'image n'a pas encore été généralisée.

Arrêter cette VM après vérification.

**Questions :**

1. Pourquoi `--import` remplace-t-il ici une installation depuis une ISO ?
2. Pourquoi cette capture est-elle une image d'OS, contrairement à un QCOW2 vide ?
3. Quels éléments seraient nécessaires si la VM possédait plusieurs disques, une NVRAM UEFI ou un TPM ?
4. Pourquoi ne faut-il pas lancer plusieurs copies non généralisées comme des serveurs de production ?

**Validation D :** image d'OS créée, contrôlée, documentée et testée par démarrage d'une nouvelle VM.

# Partie E — Construire un template généralisé · 55 min

## Exercice 9 — Préparer le socle commun

Le template doit fournir les éléments communs ; les noms d'hôte, identités et paramètres propres à chaque instance seront définis lors du déploiement.

Démarrer uniquement la VM source.

**DANS LA VM SOURCE :**

- vérifier que `cloud-init`, `qemu-guest-agent` et le serveur SSH sont installés ;
- vérifier que la configuration réseau utilise DHCP sans dépendance à la MAC source ;
- supprimer les données pédagogiques propres à une instance qui ne doivent pas être reproduites ;
- ne conserver aucun secret personnel ni clé privée utilisateur.

Préparer la découverte du support NoCloud pour ce laboratoire :

```
sudo mkdir -p /etc/cloud/cloud.cfg.d

cat <<'EOF' | sudo tee /etc/cloud/cloud.cfg.d/99-z-tp.cfg
datasource_list: [ NoCloud, None ]
preserve_hostname: false
ssh_deletekeys: true
ssh_genkeytypes: [rsa, ecdsa, ed25519]
EOF

sudo rm -f /etc/cloud/cloud-init.disabled
cloud-init --version
cloud-init clean --help
```

Vérifier aussi que cloud-init n'est pas désactivé dans la ligne de commande du noyau ou par une autre configuration de l'installateur. La restriction à NoCloud est propre au TP ; un template destiné à un autre cloud doit utiliser les sources de données adaptées.

**Contrôle réseau :**

- Ubuntu : inspecter `/etc/netplan/*.yaml`. La configuration doit utiliser DHCP sur l'interface VirtIO, sans `match` imposant la MAC de la source.
- Rocky : inspecter `nmcli connection show` puis le profil actif. Éviter une MAC imposée et des paramètres IP statiques hérités.
- Si une liaison à la MAC ou une configuration statique est présente, la corriger avec l'enseignant avant d'arrêter la VM. Ne pas supprimer tous les profils réseau sans solution de remplacement.

Nettoyer l'état cloud-init et son ancienne source locale :

```
sudo cloud-init clean --logs --seed
sudo poweroff
```

Si `--seed` n'est pas pris en charge par la version installée, ne pas ignorer le problème : vérifier l'aide et faire adapter la procédure pour retirer l'ancien seed. Le `machine-id` sera traité hors ligne à l'étape suivante.

**Ne pas redémarrer la source entre ce nettoyage et la copie destinée au template**, car cela recréerait un état d'instance.

## Généraliser une copie, jamais l'original

**SUR L'HÔTE**, confirmer l'arrêt de la source, puis :

```
sudo virsh -c qemu:///system domstate "$VM"

sudo qemu-img convert -p -f qcow2 -O qcow2 "$DISK" "$TEMPLATE"

sudo virt-sysprep --list-operations
```

Lire la liste et vérifier la présence des opérations utilisées ci-dessous. Elles sont explicites pour rendre le nettoyage prévisible :

```
sudo virt-sysprep \
  -a "$TEMPLATE" \
  --operations machine-id,ssh-hostkeys,udev-persistent-net,logfiles,tmp-files,bash-history

sudo qemu-img check -f qcow2 "$TEMPLATE"
sudo qemu-img info "$TEMPLATE"

sudo sha256sum "$TEMPLATE" \
  | tee "$HOME/tp1-kvm/preuves/template.sha256"
```

**Interpréter le nettoyage :**

- `machine-id` : retire l'identité du système pour permettre sa régénération ;
- `ssh-hostkeys` : retire les clés SSH hôte pour qu'elles soient régénérées ;
- `udev-persistent-net` : nettoie certaines anciennes associations réseau ; cela ne garantit pas le nettoyage de toutes les configurations Netplan ou NetworkManager ;
- `logfiles`, `tmp-files`, `bash-history` : retirent des traces sélectionnées.

Cette liste n'est pas une procédure universelle d'effacement des secrets. Elle ne supprime pas automatiquement tous les comptes, mots de passe, clés utilisateur, certificats applicatifs ou

identifiants de logiciels. Le template de ce TP reste un artefact de laboratoire tant qu'il n'a pas fait l'objet d'un audit adapté.

Dans ce TP, le compte pédagogique local est conservé pour l'accès console et doit être documenté. Pour une référence diffusée, définir une politique de suppression ou de verrouillage des comptes initiaux ; ne pas distribuer un compte d'administration avec un mot de passe partagé.

### Versionner la référence

Créer une fiche `template-v1` indiquant :

- OS, architecture et date ;
- logiciels communs ;
- firmware et matériel virtuel attendus ;
- nettoyage effectué ;
- comptes conservés et restrictions de diffusion ;
- personnalisation attendue au premier démarrage ;
- somme de contrôle ;
- résultats des tests de déploiement.

**Ne jamais utiliser le template lui-même comme disque système inscriptible.** Une nouvelle version se prépare dans une VM de construction, puis est arrêtée, nettoyée, exportée et testée.

**Validation E :** référence distincte de la source, nettoyée, versionnée et prête à être testée.

## Partie F — Déployer deux VM depuis le template · 65 min

### Exercice 10 — Personnaliser sans réinstaller l'OS

Cette étape utilise une configuration cloud-init minimale. L'étude détaillée de cloud-init pourra être poursuivie au TP2.

**Objectif :** créer `app01` et `app02` à partir de la même référence, sans réinstallation et sans reproduire les identités de la source.

### 10.1 Préparer une clé utilisateur sur l'hôte

Réutiliser une clé publique existante ou créer une clé dédiée :

```
# Ne pas écraser une clé existante.
ssh-keygen -t ed25519 -f "$HOME/.ssh/tp1_kvm" -C "tp1-kvm"
```

Choisir une phrase secrète. La clé privée reste sur l'hôte. Seule la clé publique sera placée dans le support de configuration.

## 10.2 Créer une copie et un support de configuration par instance

**SUR L'HÔTE**, exécuter le bloc suivant une première fois avec `N=app01`, puis une deuxième fois avec `N=app02`.

```
export N=app01
# Deuxième exécution : export N=app02

export WORK="$HOME/tp1-kvm/seed-$N"
export INSTANCE_DISK="$IMGDIR/${PREFIX}-$N.qcow2"
export SEED="/var/lib/libvirt/boot/${PREFIX}-$N-seed.iso"
export PUBKEY="$(cat "$HOME/.ssh/tp1_kvm.pub")"

mkdir -p "$WORK"

# Le disque de sortie et le nom de VM doivent être nouveaux.
sudo qemu-img convert -p -f qcow2 -O qcow2 \
  "$TEMPLATE" "$INSTANCE_DISK"

# Renouveler la graine aléatoire dans chaque copie, sans modifier la référence.
sudo virt-sysprep --enable customize -a "$INSTANCE_DISK"

cat > "$WORK/meta-data" <<EOF
instance-id: tp1-${PREFIX}-${N}-v1
local-hostname: $N
EOF

cat > "$WORK/user-data" <<EOF
#cloud-config
preserve_hostname: false
hostname: $N
manage_etc_hosts: true
ssh_deletekeys: true
ssh_genkeytypes: [rsa, ecdsa, ed25519]
ssh_pwauth: false
disable_root: true
users:
  - default
  - name: etudiant
    groups: [adm]
    sudo: ["ALL=(ALL) NOPASSWD:ALL"]
    shell: /bin/bash
    lock_passwd: true
    ssh_authorized_keys:
      - $PUBKEY
write_files:
  - path: /etc/tp-instance
    permissions: '0644'
    content: |
      Instance : $N
      Reference : ${PREFIX}-template-v1
EOF

genisoimage \
  -output "$WORK/seed.iso" \
  -volid cidata \
  -joliet -rock \
  "$WORK/user-data" "$WORK/meta-data"

sudo cp "$WORK/seed.iso" "$SEED"
```

Le compte `etudiant` est ici un compte d'administration **de laboratoire**, accessible par clé, avec une règle sudo sans mot de passe. Ce choix doit être adapté avant tout usage réel.

Le fichier `meta-data` donne une identité d'instance distincte. Le fichier `user-data` indique la personnalisation. L'ISO `cidata` transporte cette configuration ; **ce n'est pas une ISO d'installation de Linux**.

### 10.3 Créer la VM depuis le disque préparé

Toujours avec les variables de l'instance courante :

```
sudo virt-install \
  --connect qemu:///system \
  --name "${PREFIX}-$N" \
  --memory 2048 \
  --vcpus 2 \
  --import \
  --disk path="$INSTANCE_DISK",format=qcow2,bus=virtio \
  --disk path="$SEED",device=cdrom,readonly=on \
  --network network=default,model=virtio \
  --channel unix,target_type=virtio,name=org.qemu.guest_agent.0 \
  --os-variant "$OSVAR" \
  --graphics vnc,listen=127.0.0.1 \
  --noautoconsole
```

Conserver le même mode de firmware que la source. Ouvrir la console, attendre la fin du premier démarrage et relever l'adresse IP. Utiliser les baux DHCP ou l'agent invité si nécessaire.

Depuis l'hôte :

```
ssh -i "$HOME/.ssh/tp1_kvm" etudiant@ADRESSE_IP
```

**DANS CHAQUE INSTANCE :**

```
sudo cloud-init status --wait
sudo cloud-init status --long
hostnamectl --static
cat /etc/machine-id
ip -br addr
cat /etc/tp-instance
cat /opt/tp1/version.txt
sudo ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Un état comportant des erreurs ou des avertissements doit être expliqué et corrigé, pas masqué.

### 10.4 Prouver la généralisation

| Vérification | app01 | app02 | Critère de réussite |
|---|---|---|---|
| Nom libvirt et UUID | À relever | À relever | Différents |
| Disque système | À relever | À relever | Chemins distincts, sans backing file |
| Adresse MAC | À relever | À relever | Différente |
| Nom d'hôte Linux | À relever | À relever | app01 / app02 |
| machine-id | À relever | À relever | Différent et non vide |

| Vérification | app01 | app02 | Critère de réussite |
|---|---|---|---|
| Empreinte SSH hôte | À relever | À relever | Différente |
| Identité cloud-init | À relever | À relever | instance-id distinct |
| Fichier `/etc/tp-instance` | À relever | À relever | Personnalisé |
| Fichier commun `/opt/tp1/version.txt` | À relever | À relever | Même version v1 |
| Réseau | À relever | À relever | Pas de conflit lors d'un fonctionnement simultané |

Pour relever les `instance-id`, consulter les métadonnées de chaque support et les données d'instance disponibles dans `/run/cloud-init/instance-data.json`.

Vérifier aussi que le template n'a pas été modifié :

```
sudo sha256sum -c "$HOME/tp1-kvm/preuves/template.sha256"
```

**Questions :**

1. Qu'est-ce qui est commun aux deux instances ? Qu'est-ce qui doit être unique ?
2. Pourquoi leur clé publique utilisateur peut-elle être identique, alors que leurs clés SSH hôte doivent différer ?
3. Pourquoi le nom de VM passé à libvirt ne suffit-il pas à changer le hostname Linux ?
4. Quel problème introduirait la réutilisation du même `instance-id` sur un disque déjà initialisé ?
5. En quoi ce déploiement est-il plus adapté à une série de serveurs qu'un clonage non généralisé ?

**Validation F :** deux VM opérationnelles, identités distinctes, logiciels communs conservés et template inchangé.

# Partie G — Comprendre les formats d'images · 30 min

## Exercice 11 — Créer, inspecter et convertir RAW/QCOW2

Ces manipulations utilisent des **disques vides**, distincts des images d'OS créées précédemment.

**SUR L'HÔTE :**

```
mkdir -p "$HOME/tp1-kvm/formats"
cd "$HOME/tp1-kvm/formats"

qemu-img create -f raw disque-raw.img 5G
qemu-img create -f qcow2 disque-qcow2.qcow2 5G

qemu-img info disque-raw.img
```

```
qemu-img info disque-qcow2.qcow2

ls -lh disque-*
du -h disque-*

qemu-img convert -p -f raw -O qcow2 \
  disque-raw.img disque-converti.qcow2

qemu-img info disque-converti.qcow2
qemu-img compare -f raw -F qcow2 \
  disque-raw.img disque-converti.qcow2
```

La comparaison doit confirmer l'égalité du contenu logique, même si les fichiers et leurs formats diffèrent.

| Notion | Explication attendue |
|---|---|
| Taille virtuelle | Capacité du disque présentée à la VM |
| Taille apparente du fichier | Taille logique du fichier sur l'hôte, visible notamment avec `ls -lh` |
| Occupation réelle | Blocs effectivement alloués sur le stockage, observables avec `du` |
| Thin provisioning | Allocation progressive ; il faut néanmoins surveiller l'espace disponible |
| RAW | Format simple ; peut être sparse selon sa création et le système de fichiers |
| QCOW2 | Format avec métadonnées, allocation dynamique et fonctionnalités telles que backing files et snapshots internes |

**Questions :**

1. Pourquoi un RAW de 5 Gio peut-il occuper bien moins de 5 Gio sur l'hôte ?
2. Pourquoi « RAW occupe toujours toute sa capacité » est-il faux ?
3. Pourquoi le format QCOW2 ne signifie-t-il pas qu'un OS est installé ?
4. Convertir une image la généralise-t-il ?
5. Un template peut-il être stocké en RAW ? Justifier.
6. Pourquoi la conversion d'un disque système doit-elle être réalisée hors ligne dans ce TP ?

**Validation G :** sorties réelles commentées, sans inventer de tailles attendues. Les résultats varient selon le système de fichiers et les options de stockage.

# Partie H — Choisir la bonne méthode · 20 min

### Exercice 12 — Études de cas

Pour chaque situation, préciser : **choix principal, justification, risque et vérification à effectuer**.

1. Vous devez tester une mise à jour risquée sur une copie d'un serveur existant.
2. Vous devez fournir vingt VM Linux identiques en logiciels, mais avec des noms et identités distincts.
3. Vous devez conserver le système installé d'une VM avant une migration.
4. Vous disposez d'un fichier QCOW2 vide et devez obtenir un serveur Ubuntu opérationnel.
5. Vous voulez transmettre une base Linux préinstallée et réutilisable à plusieurs équipes.
6. Vous devez restaurer une application utilisant trois disques et une base de données.
7. Vous voulez créer rapidement cinq VM temporaires en minimisant le stockage local.
8. Vous avez cloné une VM et constatez une nouvelle MAC mais un hostname et une clé SSH hôte identiques : le clonage a-t-il échoué ?

### Repères de correction pour la synthèse

- **Cas 1 :** clone, en environnement isolé ; un snapshot peut compléter un test local mais n'apporte pas la même indépendance.
- **Cas 2 :** template généralisé avec personnalisation par instance.
- **Cas 3 :** image cohérente, configuration de la VM et dépendances ; test de restauration obligatoire.
- **Cas 4 :** installer depuis une ISO ou partir d'une image d'OS compatible ; un disque vide n'est pas amorçable par magie.
- **Cas 5 :** template ou image généralisée documentée, auditée et versionnée.
- **Cas 6 :** sauvegarde cohérente de l'ensemble, incluant les données applicatives et les éléments de configuration nécessaires ; une simple copie du disque système ne suffit pas.
- **Cas 7 :** éventuellement clones liés à une référence immuable ; expliquer la dépendance à la chaîne de backing files.
- **Cas 8 :** non, pas nécessairement : les identités matérielles et internes de l'OS ne sont pas renouvelées par les mêmes mécanismes.

# Approfondissement facultatif — Clone lié et backing file

Un clone complet possède un disque indépendant. Un clone lié utilise une image de base et un fichier ne stockant que ses différences.

**SUR L'HÔTE**, pour un fichier de sortie nouveau :

```
export OVERLAY="$IMGDIR/${PREFIX}-demo-overlay.qcow2"

sudo qemu-img create \
  -f qcow2 \
  -F qcow2 \
  -b "$TEMPLATE" \
  "$OVERLAY"

sudo qemu-img info --backing-chain "$OVERLAY"
```

Ne pas démarrer cette image sans une procédure de personnalisation analogue à celle des instances précédentes.

**À comprendre :**

- le gain de stockage s'accompagne d'une dépendance ;
- la base ne doit plus être modifiée ni supprimée ;
- déplacer seulement l'overlay ne déplace pas toute la VM ;
- aplatir l'overlay avec `qemu-img convert`, hors ligne, permet de produire une image autonome.

Cet exercice ne remplace pas les clones complets demandés dans le parcours principal.

## Dépannage raisonné

| Symptôme | Vérifications prioritaires |
|---|---|
| Extensions CPU non visibles | UEFI/BIOS, virtualisation imbriquée, architecture |
| Connexion libvirt impossible | Connexion système, service ou sockets, droits |
| Réseau `default` absent | Définition du réseau fournie par l'enseignant |
| ISO ou disque inaccessible | Chemin, permissions, contexte SELinux/AppArmor |
| Variante OS inconnue | Liste `virt-install --osinfo list`, données OS installées |
| `shutdown` lancé mais disque encore utilisé | État effectif de la VM ; attendre l'arrêt |
| `domifaddr` vide | Agent invité et canal, baux DHCP, console |
| Même IP après duplication | Client-id/DUID, machine-id, profil réseau et baux DHCP |
| Personnalisation cloud-init absente | Volume `cidata`, fichiers à sa racine, activation cloud-init, ancienne seed et instance-id |
| Identités identiques après déploiement | Nettoyage réellement effectué, utilisation d'une copie fraîche, logs cloud-init |
| Agent invité non démarré | Présence du canal VirtIO et état du service |

| Symptôme | Vérifications prioritaires |
|---|---|
| Disque réimporté non amorçable | Firmware, ordre de démarrage, contrôleur, partitions et dépendances |
| Erreur de clé SSH après réattribution d'une IP | Vérifier d'abord l'empreinte en console ; ne pas supprimer aveuglément l'alerte de sécurité |

Dans les instances, utiliser notamment :

```
sudo cloud-init status --long
sudo journalctl -b --no-pager
sudo tail -n 80 /var/log/cloud-init.log
sudo tail -n 80 /var/log/cloud-init-output.log
```

## Livrables

Remettre un compte rendu structuré comprenant :

- description du poste hôte et du parcours choisi ;
- diagnostic KVM/libvirt ;
- commande de création et configuration de la VM ;
- preuves d'administration ;
- tableau source/clone avec valeurs observées et test d'indépendance des disques ;
- fiche de l'image d'OS et preuve de sa réimportation ;
- fiche du template, nettoyage expliqué et somme de contrôle ;
- tableau app01/app02 démontrant les identités distinctes ;
- comparaison RAW/QCOW2 basée sur les sorties obtenues ;
- réponses argumentées aux cas d'usage ;
- difficultés rencontrées, diagnostic et solution ;
- conclusion personnelle : « Je choisis un clone quand…, un template quand…, une image quand… ».

**Conserver localement les gros fichiers QCOW2/RAW.** Ne déposer ni clé privée, ni secret, ni image contenant des données personnelles. Fournir les fichiers volumineux uniquement sur demande de l'enseignant.

## Évaluation sur 20 points

| Critère | Points | Condition de validation |
|---|---|---|
| Installation et diagnostic KVM/libvirt | 2 | Environnement fonctionnel et diagnostic expliqué |
| Création et administration de VM | 4 | VM démarrable, réseau et cycle de vie maîtrisés |
| Clone complet et analyse des identités | 3 | Indépendance des disques et observations justes |

| Critère | Points | Condition de validation |
|---|---|---|
| Image d'OS et test de réimportation | 3 | Système installé réellement récupéré |
| Template et déploiement de deux instances | 5 | Généralisation prouvée par des identités distinctes |
| Formats et choix des usages | 2 | Explications et choix argumentés |
| Qualité et reproductibilité du compte rendu | 1 | Preuves, commandes et difficultés documentées |
| **Total** | **20** | |

**Critères essentiels :** un fichier produit mais non testé ne valide pas la compétence correspondante ; un template n'est pas validé sur son seul nom de fichier ; les différences d'identité doivent être démontrées sur les instances déployées.

## Validation individuelle finale

Sans recopier une réponse préparée, l'étudiant doit pouvoir :

- ☐ créer et démarrer une VM ;
- ☐ expliquer le rôle de KVM, QEMU et libvirt ;
- ☐ cloner une VM arrêtée et identifier ce qui a été copié ;
- ☐ distinguer UUID libvirt, MAC, hostname, machine-id et clé SSH hôte ;
- ☐ capturer un système installé et le réimporter ;
- ☐ expliquer pourquoi une ISO, un disque vide et une image d'OS ne sont pas équivalents ;
- ☐ préparer un template sans modifier l'unique copie de la source ;
- ☐ déployer deux instances personnalisées sans réinstaller Linux ;
- ☐ choisir entre clone, template et image selon un besoin concret ;
- ☐ expliquer les limites d'une image seule comme moyen de sauvegarde.

## Ressources officielles

- Ubuntu Server : https://ubuntu.com/download/server
- Rocky Linux : https://rockylinux.org/download
- libvirt : https://libvirt.org/
- virt-sysprep : https://libguestfs.org/virt-sysprep.1.html
- cloud-init — commandes : https://docs.cloud-init.io/en/latest/reference/cli.html
- cloud-init — NoCloud : https://docs.cloud-init.io/en/latest/reference/datasources/nocloud.html
- Manuels locaux : `man virt-install`, `man virt-clone`, `man virsh`, `man qemu-img`, `man virt-sysprep`.

**Transition vers le TP2 :** automatiser davantage la personnalisation cloud-init, les comptes, les clés SSH, le réseau et les rôles applicatifs à partir du template validé.